



Distributed Telemetry Engine

Scalable Distributed Computing - Project Report

Giacomo Cesaretti, Matthew Bernardi

A.Y. 2025/2026

Contents

1	Introduction	3
2	Technologies	3
3	System Architecture	4
3.1	Overview Architecture	4
3.2	Event Processing Flow	5
3.3	Monitoring and Reconfiguration Architecture	6
3.3.1	Reconfiguration Planner Algorithm	6
4	Fault Tolerance	7
4.1	Idempotency and Logical Exactly-Once Effect	7
4.2	Invalid Message Handling (Dead Letter Queue)	8
4.3	Processor Crashes and Rebalancing	8
5	Deployment & Infrastructure Setup	8
6	Implementation Choices and Rationale	9
6.1	Kafka and Parallel Processing	9
6.2	Reliability Choices	9
6.3	Temporal Aggregation	10
6.4	Runtime Metrics and Reconfiguration Parameters	10
6.5	Event Generator	10
6.6	Deployment Choices	11
7	Reconfiguration Design	11
7.1	Scope and Design Goals	11
7.2	Target Reconfiguration Architecture	11
7.3	Automatic Processor Scaling	12
7.4	Dynamic Kafka Partition Scaling	13
7.4.1	Partition Scaling Limitations	13
7.5	Kafka Broker Scaling	14
7.6	Control Plane Failover	15
7.7	Reconfiguration Safety and Failure Handling	16
7.8	Design Limitations	16
8	Testing	16
9	Conclusion	17

1 Introduction

This project concerns the design and partial implementation of a distributed platform for the collection and processing of telemetry events generated by multiple sources, such as sensors and software applications. Its goal is to build a scalable and fault-tolerant pipeline capable of ingesting telemetry data, distributing the workload across multiple processing instances, computing time-based aggregations, and storing the results for later analysis.

The system follows an event-driven architecture. Telemetry events are received through an API and published to Apache Kafka, which acts as the messaging backbone of the platform. From there, the events are consumed by multiple Processor instances belonging to the same consumer group, allowing the workload to be distributed and processed in parallel. Each Processor computes temporal aggregations over predefined time windows and stores the resulting aggregates in PostgreSQL.

A central requirement is robustness in the presence of failures and message redelivery. Since retries, duplicate deliveries, and Processor crashes may occur, the system adopts an idempotent processing strategy based on event deduplication, ensuring that each event produces only one logical effect on the final aggregates.

The system also includes a simple Reconfiguration Planner that observes the current load and suggests whether the number of Processor instances should be increased. This component represents a first step toward adaptive scaling.

The implemented prototype includes the ingestion API, Kafka integration, multiple Processor instances, PostgreSQL storage, event deduplication, time-windowed aggregations, runtime metrics collection, the Reconfiguration Planner, and a Docker Compose-based test and deployment environment. Runtime infrastructure reconfiguration—including automatic Processor scaling, Kafka partition and broker scaling, and control-plane failover—is instead addressed at the design level and left as a future extension.

Overall, the project provides a practical example of asynchronous communication, parallel processing, fault tolerance, idempotence, persistence, and elasticity in distributed systems.

2 Technologies

The main technologies used while developing the project are:

Docker and Docker Compose. Used to containerize the application and define a reproducible multi-service development and deployment environment.

Apache Kafka. Message broker used to store telemetry events in topics and distribute partitions among Processors belonging to the same consumer group. The Python client used by the project is `confluent-kafka`.

PostgreSQL. Relational database used to store temporal aggregates and processed event identifiers for deduplication.

FastAPI & Uvicorn. Used to implement the REST API and run the web service.

Pydantic. Used to validate JSON payloads against the application’s data models.

HTTPX. Used by the Event Generator to perform asynchronous HTTP requests.

Pytest. Used to implement and execute unit, integration, and end-to-end tests.

3 System Architecture

3.1 Overview Architecture

The system is composed of an ingestion layer, a Kafka-based messaging layer, a distributed processing layer, and a persistent storage layer. In addition, a monitoring and reconfiguration path observes the runtime state of the system and produces scaling recommendations.

Figure 1 provides an overview of the complete architecture and of the interactions between its main components.

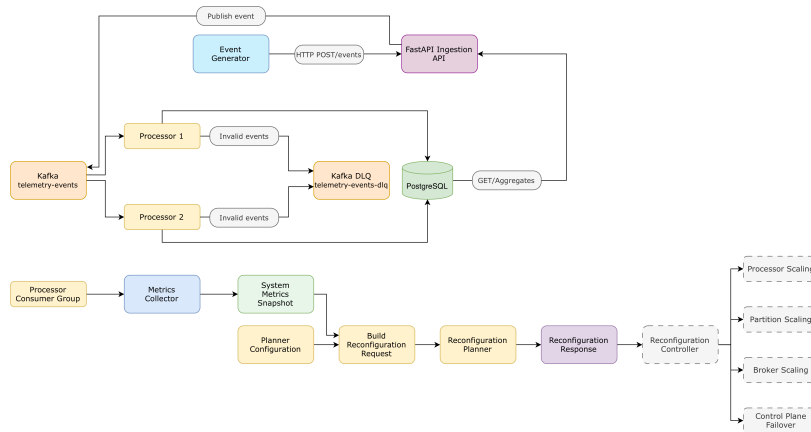


Figure 1: Overall system architecture.

3.2 Event Processing Flow

While the previous figure presents the static organization of the system, the following sequence diagram illustrates its runtime behaviour when a telemetry event is received.

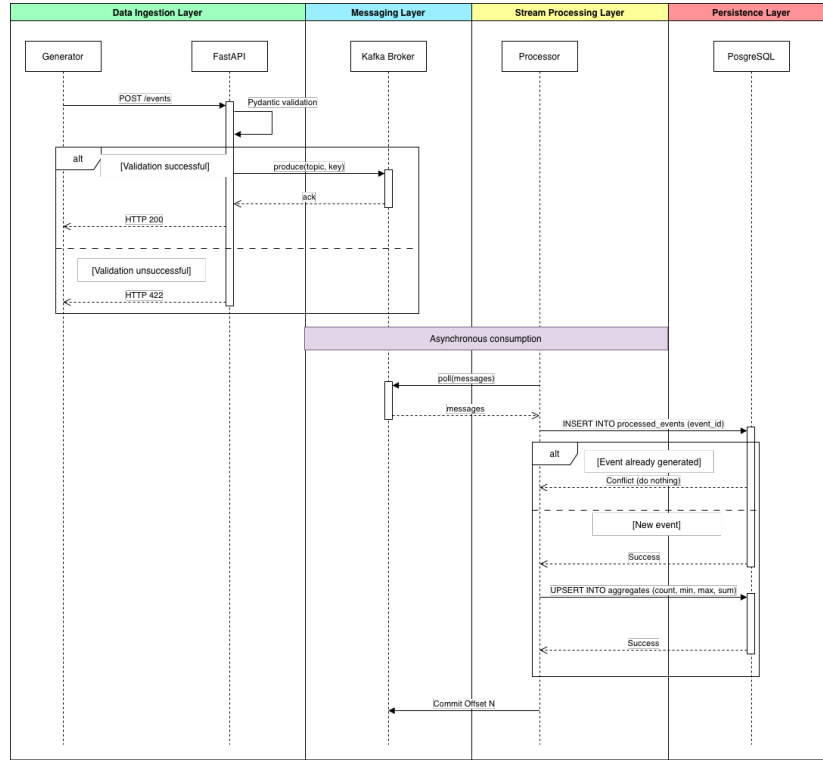


Figure 2: Sequence Diagram for the Event Processing phase.

The main layers are:

Data Ingestion Layer. The Generator submits an event through the `POST /events` endpoint exposed by FastAPI. The payload is validated through Pydantic before being accepted by the API and handed to the Kafka producer.

Messaging Layer. Kafka contains the `telemetry-events` topic for valid telemetry traffic and the `telemetry-events-dlq` topic for malformed events. Messages in the main topic are partitioned using `source_id` as the key.

Stream Processing Layer. Processor instances belong to the same Kafka consumer group, `adaptive-processors`. Kafka assigns partitions among the active members and rebalances them when group membership changes.

Persistence Layer. PostgreSQL contains the `processed_events` table, used for deduplication, and the `aggregates` table, which stores count, sum, minimum, and maximum values for each temporal aggregation window.

3.3 Monitoring and Reconfiguration Architecture

In addition to the main event-processing pipeline, the system implements a separate monitoring path. Its purpose is to observe the current Kafka and consumer-group state and provide the Reconfiguration Planner with the information required to evaluate the available processing capacity.

Figure 3 shows this monitoring path, while Figure 4 shows the related sequence diagram.

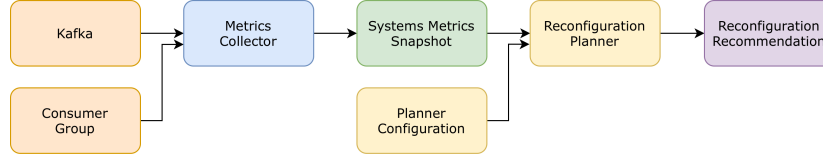


Figure 3: Monitoring and reconfiguration decision path.

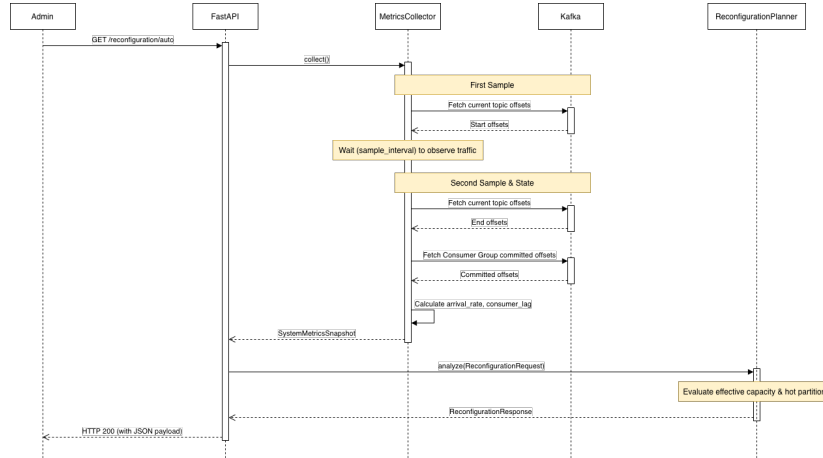


Figure 4: Sequence Diagram for the Reconfiguration.

3.3.1 Reconfiguration Planner Algorithm

The `ReconfigurationPlanner` implements a stateless decision-making algorithm based on two aspects: overall processing capacity and partition load distribution.

- **Processor Scaling Logic:**

- The planner calculates the effective capacity of one Processor as `worker_capacity` multiplied by `target_utilization`.
- The number of required workers is computed as:

$$\text{required_workers} = \left\lceil \frac{\text{arrival_rate}}{\text{effective_capacity}} \right\rceil$$

- When partition information is available, the number of useful required workers is capped by the current number of Kafka partitions, since additional consumers would not provide further Kafka-level parallelism:

$$\text{useful_required_workers} = \min(\text{required_workers}, \text{partition_count})$$

The target configuration is then computed without scaling down the currently active Processor pool.

- If the required number exceeds the current number of workers, the planner generates a `SCALE_WORKERS` action. Consumer lag is used as additional evidence in the explanation of the recommendation.

- **Hot Partition Detection:**

- The planner computes the average incoming rate across partitions.
- A partition is considered potentially hot when its rate exceeds the average multiplied by the configurable `HOT_PARTITION_FACTOR` (1.5 by default).
- In that case, the planner produces an `INVESTIGATE_HOT_PARTITION` action.

4 Fault Tolerance

The implemented fault-tolerance mechanisms focus on Processor failures, message redelivery, and malformed events. They combine Kafka consumer-group rebalancing with explicit offset management and database-level idempotency.

4.1 Idempotency and Logical Exactly-Once Effect

Kafka consumption follows at-least-once semantics because automatic commits are disabled and each Processor commits an offset only after the corresponding message has been successfully handled.

To avoid duplicate logical effects, each event has a unique `event_id`. During processing, the identifier is inserted into the `processed_events` table using `ON`

`CONFLICT (event_id) DO NOTHING`. If the insertion succeeds, the corresponding aggregate is updated through an UPSERT in the same PostgreSQL transaction; otherwise, the event is recognized as a duplicate and does not modify the aggregate.

As a consequence, an event may be delivered more than once by Kafka, but each `event_id` contributes at most once to the final aggregate.

4.2 Invalid Message Handling (Dead Letter Queue)

Malformed events are redirected to the dedicated `telemetry-events-dlq` topic instead of being discarded or repeatedly retried. The original payload and key are preserved, together with metadata about the original topic, partition, and offset. The source offset is committed only after the DLQ publication succeeds.

Under normal operation, the DLQ is not expected to receive messages coming through the FastAPI ingestion path. The API validates each incoming JSON payload against the `TelemetryEvent` model using Pydantic before publishing it to Kafka. If the payload is malformed, for example because required fields are missing, unexpected fields are present, or field values do not satisfy the model constraints, the request is rejected with an HTTP 422 response and the event is not published to the `telemetry-events` topic.

The DLQ therefore acts as an additional defensive mechanism at the Processor level. It is used when an invalid message reaches Kafka without passing through the ingestion API, for example when a producer publishes directly to the `telemetry-events` topic. In this case, the Processor performs a second validation against the same `TelemetryEvent` model and redirects invalid messages to the `telemetry-events-dlq` topic.

This behaviour is explicitly verified by the project tests, although direct publication of malformed events to Kafka is not part of the normal operating flow.

4.3 Processor Crashes and Rebalancing

Processor instances belong to the same Kafka consumer group. If one Processor becomes unavailable, Kafka detects the membership change and triggers a consumer-group rebalance, assigning the affected partitions to the remaining healthy Processor instances. This allows event processing to continue without manual partition reassignment.

5 Deployment & Infrastructure Setup

The entire distributed system is containerized using Docker and orchestrated through Docker Compose, which defines the network topology, environment variables, startup dependencies, and service instances.

The main infrastructure choices are:

- **Kafka in KRaft Mode:** Kafka runs without ZooKeeper, keeping the local deployment compact.
- **Automated Topic Provisioning:** An ephemeral `kafka-init` service creates the `telemetry-events` topic with four partitions and the `telemetry-events-dlq` topic before the application services start.
- **Database Initialization:** PostgreSQL mounts an `init.sql` script that creates the `processed_events` and `aggregates` tables on first startup.
- **Application Service Orchestration:** The Python components are built from a shared `Dockerfile`. Docker Compose starts one API instance and two Processor instances. Startup dependencies and health checks ensure that the required infrastructure is available before the application services begin processing.

The environment can therefore be launched reproducibly with a single Docker Compose command.

6 Implementation Choices and Rationale

This section summarizes the main implementation choices that require explicit justification. Numerical values such as partition count, sampling interval, and planner thresholds are configurable parameters selected to make the prototype behaviour observable during testing rather than universally optimal settings.

6.1 Kafka and Parallel Processing

The `telemetry-events` topic uses four partitions, while the current deployment contains two Processor instances. Four partitions provide parallelism for the current deployment while leaving room for additional Processors.

The `source_id` is used as the Kafka message key so that events from the same source are consistently routed according to Kafka partitioning. Since one partition can be assigned to at most one consumer of the same group, useful Processor parallelism is bounded by the current partition count.

The prototype uses a single Kafka broker with replication factor one. This is a deliberate simplification of the local Docker Compose environment; broker replication and failover belong to the designed, but not implemented, reconfiguration architecture.

6.2 Reliability Choices

Manual Kafka offset commits and PostgreSQL-based deduplication were selected to combine at-least-once delivery with an idempotent logical effect. Details of the mechanism are described in the Fault Tolerance section.

The DLQ follows the same principle: invalid messages are acknowledged only after they have been successfully redirected, preventing silent loss of malformed events.

6.3 Temporal Aggregation

Events are grouped into fixed one-minute windows using:

```
source_id + metric_name + window_start
```

A one-minute window is short enough to make aggregation behaviour observable during experiments while still grouping multiple events into meaningful buckets.

For each window, the system stores count, sum, minimum, and maximum. The average is derived as `sum / count`, avoiding redundant stored state.

6.4 Runtime Metrics and Reconfiguration Parameters

The Metrics Collector uses Kafka administrative APIs rather than joining the Processor consumer group, so monitoring does not alter partition assignment.

It collects overall arrival rate, per-partition arrival rates, consumer lag, and the number of active Processors. The default sampling interval is five seconds, chosen as a compromise between responsiveness and measurement stability.

The Planner estimates the required number of workers as:

$$N_{\text{required}} = \left\lceil \frac{\text{arrival_rate}}{C_{\text{worker}} \cdot U_{\text{target}}} \right\rceil$$

The prototype uses `worker_capacity = 100 events/s` and `target_utilization = 0.8`. The 80% target leaves processing headroom instead of planning continuous operation at the assumed maximum capacity.

A partition is considered potentially hot when its arrival rate exceeds 1.5 times the average partition rate. This factor is a simple heuristic chosen to make significant skew visible without introducing a more complex statistical model.

6.5 Event Generator

The synthetic Event Generator supports uniform and skewed traffic distributions. In the skewed case, one source generates 60% of the traffic, deliberately creating a visible imbalance for hot-partition testing.

The Generator can also resend previously generated events using the same `event_id`, allowing the idempotency mechanism to be tested under duplicate-delivery conditions.

6.6 Deployment Choices

Docker Compose provides a reproducible local environment. Kafka and PostgreSQL health checks coordinate startup, PostgreSQL uses a persistent volume, and the API and Processor containers use the `restart: unless-stopped` policy to recover from unexpected process termination.

7 Reconfiguration Design

7.1 Scope and Design Goals

The implemented Reconfiguration Planner analyzes runtime metrics and produces recommendations about the required number of Processor instances. It does not directly modify the running infrastructure.

The target design extends this architecture with mechanisms for automatic Processor creation, Kafka partition and broker scaling, and control-plane failover.

The main design goals are:

- **Scalability:** increase processing capacity as workload grows;
- **Availability:** avoid unnecessary interruption during reconfiguration;
- **Safe reconfiguration:** apply and verify infrastructure changes in a controlled way;
- **Fault tolerance:** recover from failures during reconfiguration;
- **Separation of concerns:** keep decision-making in the Planner separate from infrastructure management.

7.2 Target Reconfiguration Architecture

The target architecture introduces a *Reconfiguration Controller* between the Planner and the infrastructure.

The Planner defines the desired configuration, while the Controller is responsible for comparing it with the actual infrastructure state and applying only the changes required to reach that configuration.

The Controller interacts with three logical interfaces:

- **Container management interface:** manages Processor instances;
- **Kafka administration interface:** manages partitions, brokers and replica placement;
- **Control-plane coordination interface:** coordinates Controller instances and persists reconfiguration state.

The general workflow is:

1. observe the current system state;
2. obtain the desired configuration from the Planner;
3. compare desired and actual state;
4. execute the required infrastructure operations;
5. verify the resulting state;
6. mark the operation as completed or failed.

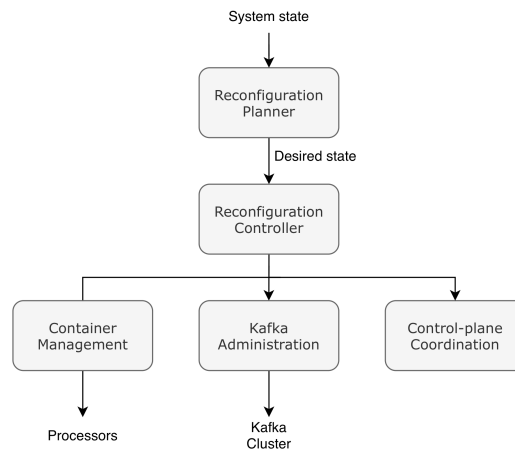


Figure 5: Reconfiguration Controller

7.3 Automatic Processor Scaling

When the Planner recommends a higher number of Processor instances, the Controller compares the desired worker count with the currently active instances and starts the missing Processors.

Each new Processor joins the same Kafka consumer group as the existing ones. Kafka then performs a consumer-group rebalance and automatically redistributes the available partitions.

The scaling procedure is:

1. obtain the current and desired Processor counts;
2. create the missing Processor instances;
3. verify that the new instances are running and have joined the consumer group;

4. allow Kafka to rebalance partition assignments;
5. verify the resulting configuration.

If creation or startup fails, the operation can be retried. Before retrying, the Controller reads the actual infrastructure state so that already-created instances are not duplicated.

The useful level of Processor parallelism remains bounded by the number of Kafka partitions, motivating the partition-scaling mechanism described next.

7.4 Dynamic Kafka Partition Scaling

If the desired Processor parallelism exceeds the current number of Kafka partitions, the Controller may increase the partition count of the `telemetry-events` topic.

The operation is expressed in terms of a desired partition count rather than as an unconditional “add partitions” command. This allows the Controller to compare the target with the actual configuration before executing or retrying the operation.

The procedure is:

1. read the current partition count;
2. determine whether additional partitions are required;
3. request the new partition count through the Kafka administration interface;
4. verify the updated topic metadata;
5. allow producers and consumers to observe the new topology and Kafka to rebalance the consumer group.

7.4.1 Partition Scaling Limitations

Increasing the number of partitions does not redistribute records already stored in existing partitions; new partitions are used only for subsequent writes.

The operation may also affect key-based partitioning. Since `source_id` is used as the Kafka key, future records for the same source may map to a different partition after the partition count changes. Therefore, ordering is only assumed within an individual partition. This does not affect the implemented aggregate operations, which are independent of arrival order and protected by event-level idempotency.

Finally, reducing the number of partitions is not handled as the inverse of partition creation and would require a different migration strategy.

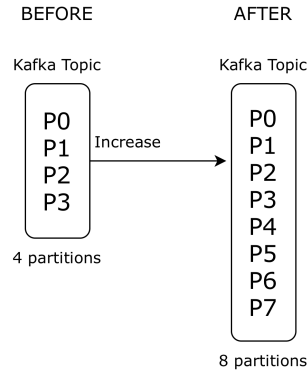


Figure 6: Example of partition scaling

7.5 Kafka Broker Scaling

Partition scaling increases logical processing parallelism but does not add physical Kafka resources. If storage, network, disk throughput, or request handling becomes a broker bottleneck, additional brokers may therefore be required.

In the target architecture, the Controller provisions a new broker and verifies that it has joined the cluster. Since adding a broker does not automatically redistribute existing partition replicas, the Controller must also trigger a replica reassignment.

The main steps are:

1. provision and start a broker with a unique identifier;
2. verify that it has joined the Kafka cluster;
3. obtain the current replica distribution;
4. compute and apply a new replica placement;
5. verify that the reassignment has completed.

Broker scaling therefore increases the physical resources of the Kafka cluster, while partition scaling primarily affects logical topic parallelism.

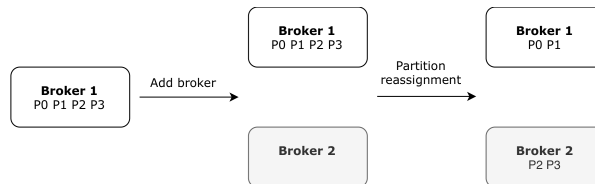


Figure 7: Example of broker scaling

7.6 Control Plane Failover

A single Reconfiguration Controller would introduce a control-plane single point of failure. The target architecture therefore considers multiple Controller instances using an active/standby model.

Only the current leader is allowed to modify the infrastructure. Desired configuration and reconfiguration progress are stored persistently so that a standby Controller can recover an interrupted operation if the leader fails.

The failover procedure is:

1. elect one Controller as active leader;
2. persist the desired state and progress of ongoing operations;
3. detect the loss of the current leader;
4. allow a standby Controller to acquire leadership;
5. compare persisted desired state with actual infrastructure state;
6. execute only the operations still required.

The coordination mechanism must guarantee exclusive leadership to prevent multiple Controllers from applying conflicting changes simultaneously.

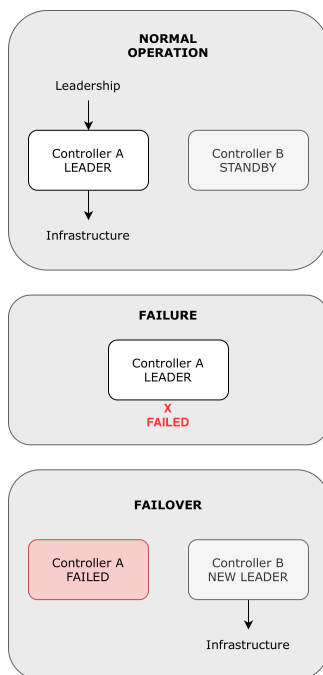


Figure 8: Failover scheme

7.7 Reconfiguration Safety and Failure Handling

A reconfiguration may fail after only part of the desired change has been applied. The Controller therefore follows a state-based reconciliation approach, comparing the desired configuration with the actual infrastructure state before deciding which operation is required.

The main principles are:

- **State verification:** inspect the actual infrastructure state before applying a change;
- **Idempotent reconfiguration:** express operations as desired state so that retries do not create duplicate changes;
- **Sequential conflicting changes:** avoid concurrent reconfigurations affecting the same resources;
- **Health verification:** consider an operation complete only after the new configuration is operational;
- **Persistent state and recovery:** persist desired state and operation progress so that execution can resume after a Controller failure.

These principles apply uniformly to Processor creation, partition scaling, broker scaling and control-plane recovery.

7.8 Design Limitations

The reconfiguration architecture intentionally remains at the design level. The implemented deployment is static: Docker Compose starts a predefined set of services, while the Planner only produces recommendations.

The prototype does not automatically create Processor containers, modify topic partition counts, provision Kafka brokers, reassign replicas, or perform control-plane leader election and failover. Policies for deciding when partitions or brokers should be added are also outside the implemented Planner.

These limitations are consistent with the project scope: event processing, monitoring, deployment and recommendation logic are implemented, while automatic infrastructure reconfiguration is left as a future extension.

8 Testing

The system was validated through automated unit and integration tests covering data-model validation, event generation, HTTP submission, Processor logic, database access, aggregation, deduplication, DLQ handling, metrics collection and the Reconfiguration Planner.

After component-level testing, a final end-to-end test validated the complete pipeline without mocks:

HTTP API → Kafka → Processor → PostgreSQL → Aggregates API

The test submits multiple events, retrieves the resulting aggregate through the API, and resends the same `event_id` to verify that duplicate delivery does not produce an additional logical effect.

A further failure test was performed by stopping one Processor instance. After Kafka completed the consumer-group rebalance, the remaining Processor continued processing the topic partitions and the same end-to-end test still completed successfully.

All automated component tests and the final end-to-end and Processor-failure validations completed successfully.

9 Conclusion

This project designed and implemented a distributed telemetry engine capable of ingesting, processing and aggregating streams of event data through an event-driven architecture based on Apache Kafka.

Consumer-group rebalancing, manual offset management and PostgreSQL transactional deduplication provide resilience to duplicate delivery and Processor crashes while preserving one logical effect per event. The final tests validated both the complete processing pipeline and continued operation after the loss of one Processor.

The Metrics Collector and Reconfiguration Planner add observability and a first form of elasticity by estimating processing needs and detecting load imbalances. The current deployment remains static, while the proposed Reconfiguration Controller describes how Processor scaling, Kafka partition and broker scaling, and control-plane failover could be automated in a future extension.